

Sequential Object Placement Optimization with Convex Decomposition

Yuezhe Zhang¹, Xiangyu Lyu¹, Sohan Rudra¹, Davide Tateo^{1,2,3} and Georgia Chalvatzaki^{1,4,5}

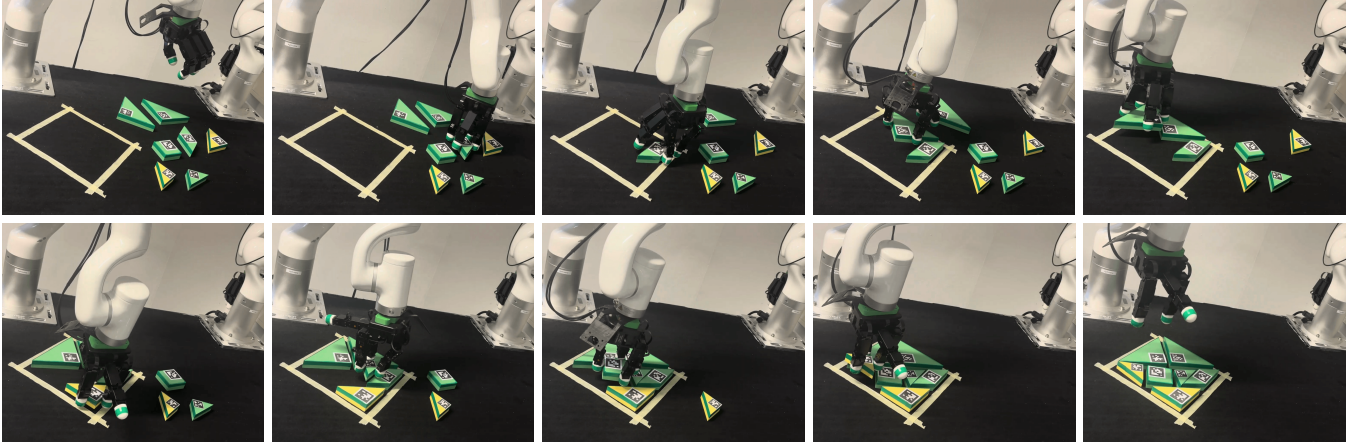


Fig. 1: A sequence of screenshots of placing the Tangram puzzle using an Allegro Hand and an Xarm.

Abstract—Robotic object packing has been a core challenge for robotic deployment in logistics, industry, etc., due to the curse of dimensionality in combinatorial search and the difficulty of dealing with dynamic and contact constraints for irregularly shaped objects. Current heuristic and learning-based methods assume a limited spatial discretization resolution of space, and computation becomes extremely inefficient as discretization accuracy increases. In this work, we eliminate these assumptions by introducing SOPO-CD, a sequential optimization framework that frames object placement as a differentiable nonlinear optimization problem in a decomposed free space. We prove that placing a convex object inside a convex hull is essentially constraining the vertices of the object inside the convex hull. The constraints and their derivatives can be written in closed form and calculated within 200ns. We implement a custom solver that achieves optimal placement within tightly constrained space in milliseconds; a $100\times$ speedup compared to a classical grid search method. We generalize our framework to 2D Tangram, 2D Tetris, and 3D Bin Packing, and have demonstrated strong computational performance and packing utility. We also demonstrate solving a real-world Tangram puzzle online using an Allegro Hand and an Xarm.

I. INTRODUCTION

Finding efficient placements of as many objects as possible within a designated space has gained multidisciplinary interests from combinatorial optimization, computational geometry, and machine learning. While existing algorithms [1], [2], [3], [4], [5] have made strides in recent years, they often assume a predefined discretized action space for both position and orientation, and the action space grows explosively with the accuracy of the discretization. Packing Configuration Trees [6], [7] is the first learning-based method that solves online 3D bin packing in a continuous space. However, it is

sample-inefficient, and the exploration-exploitation dilemma makes it difficult to generalize to non-convex shapes.

In this paper, we introduce SOPO-CD, a sequential optimization framework that frames object placement as a differentiable nonlinear optimization problem in a decomposed free space. We first divide the free space into a set of convex hulls in 2D or 3D using greedy algorithms. We prove that placing a convex object inside a convex hull is essentially constraining the vertices of the object inside the convex hull. The constraints are more efficient than checking collisions among convex object pairs. We provide first- and second-order analytic derivatives of the constraints and the Lagrangian function, resulting in 2 – 20 times faster than an AutoDiff method. By selecting a set of convex hulls in free space using heuristics and assigning the convex hull to each convex body of the object, we frame the object placement as a differentiable nonlinear optimization problem. We implement a custom Sequential Quadratic Programming (SQP) solver based on [8] to solve the optimization problem and achieve optimal placement within a tightly constrained space in milliseconds.

We evaluate SOPO-CD on 2D Tangram, 2D Tetris, and 3D Bin Packing. For Tangram, we test on a given sequence of objects and outperform SQP with a state-of-the-art differentiable collision checker, achieving a $10\times$ speedup and much higher success rates. For Tetris, we test on a random sequence of objects and achieve a packing utility of 77% with a computation time of 20ms per object in a batch of 8, a $50\times$ speedup over a grid search method. For 3D Bin Packing, we test on a random sequence of objects and achieve a packing utility of 80% with a computation time of 15ms per object in a batch of 8, a $100\times$ speedup over a grid search method. We also demonstrate a real-world Tangram puzzle task using a high-degree-of-freedom system.

¹Interactive Robot Perception & Learning, Technical University of Darmstadt, Germany; ²Intelligent Autonomous Systems, Technical University of Darmstadt, Germany; ³Robotics and Semantic Systems, Lund University, Sweden; ⁴Hessian.AI; ⁵Robotics Institute Germany.

II. RELATED WORKS

Early interest in the 3D bin packing problem focused primarily on the offline setting, where all items are known a priori and can be placed in arbitrary order. [9] solved this problem with an exact branch-and-bound approach. However, the problem is strongly NP-Hard, the exact algorithms do not guarantee optimal results within a reasonable amount of time. Therefore, heuristic methods and metaheuristic approaches have been developed to obtain approximate solutions quickly, such as the Bottom-Left (BL) heuristic [10] and the Best-Fit-Decreasing heuristic [11]. However, in many real-world application scenarios, e.g., logistics or warehousing, the upcoming items cannot be fully observed. Many online bin packing problems are solved using either heuristics or learning-based methods. Deepest-Bottom-Left-Fill (DBLF) [12] was proposed and combined with a genetic algorithm to place items in the deepest, bottom-most, left-most position. [1] performed grid search on a heightmap and proposed a Heightmap-Minimization method to minimize the volume increase of the packed items. Many RL works [2], [5] directly learn their policy on a grid by discretizing the full coordinate space, where the action space grows explosively with the discretization accuracy. Packing Configuration Trees [6], [7] is the first learning-based method that solves online 3D bin packing in a continuous space. However, the size of the packing action space scales with the number of candidate placements, which is still huge in continuous domains, and the sampling costs for training RL agents are thus expensive, making it difficult to generalize to non-convex shapes.

Therefore, we frame the object placement as an optimization problem that handles the continuous space naturally. It is crucial to consider constraints in a differentiable manner. Differentiable collision checking between two convex shapes can be formulated as a convex optimization problem [13], [14] and solved in microseconds. However, the computation time may grow as the number of collision pairs grows. In the packing problem, this is important because the objects to be placed need to check for collisions with all previously placed objects. In addition, it is challenging to do narrow-space planning with the non-convex obstacles. We thus take a different view by considering the object placement in the decomposed free space, inspired by [15], [16], [17], [18]. Some recent works also frame stacking [19], [20], [21] as an optimization problem, but they are soft constrained and cannot generalize to placing a large number of objects in a tightly constrained space.

III. PRELIMINARIES

A. Problem formulation

We study the problem of optimal object placement within a constrained space. Given an object set $\mathcal{O}$ and the collision-free space $\mathcal{C}_{free}$, we aim to find the optimal translation $t \in \mathbb{R}^3$ and rotation parameters $q \in \mathbb{R}^3$ that optimizes the objective function $f(t, q)$, such as the depth of the object, and makes the object collision free.

$$\begin{aligned} & \underset{t, q \in \mathbb{R}^3}{\text{minimize}} && f(t, q) \\ & \text{s.t.} && g(\mathcal{O}, t, q) \in \mathcal{C}_{free} \end{aligned} \quad (1)$$

where g applies the transformations $\{t, q\}$ on the object set $\mathcal{O}$. Note that the object set $\mathcal{O}$ and collision-free space $\mathcal{C}_{free}$ are not necessarily convex, which makes the problem challenging to solve.

B. Convex Hull Definition

The convex hull of a set of points $\mathcal{S} \in \mathbb{R}^d$ is the intersection of all convex sets containing $\mathcal{S}$. We define the vertices of the convex hull as $X = \{x_1, x_2, \dots, x_m\}$. There are two ways of representing the convex hull $\text{conv}(X)$.

$\mathcal{V}$ -representation writes the convex hull as a convex combination of points in $\mathbb{R}^d$. Mathematically, this reads

$$\text{conv}(X) = \left\{ x \in \mathbb{R}^d \mid x = \sum_{i=1}^m \omega_i x_i, \sum_{i=1}^m \omega_i = 1, \omega_i \geq 0 \right\} \quad (2)$$

where $\omega_i, i = 1, \dots, m$ are the convex coefficients.

Alternatively, $\mathcal{H}$ -representation defines the convex hull as a finite number of inequalities. Mathematically, this yields

$$\text{conv}(X) = \left\{ x \in \mathbb{R}^d \mid Ax \leq b \right\} \quad (3)$$

where A and b can be computed from X and they determine the halfspaces of the convex hull.

C. Differentiable Collision via LP

One state-of-the-art differentiable collision checker is proposed by [14] as solving a Linear Cone Program between a set of convex primitives. It solves for the minimum uniform scaling α that must be applied to the convex primitives $S_1(\alpha)$ and $S_2(\alpha)$ for an intersection to occur. When primitives are not in contact, the minimum scaling $\alpha > 1$, and when the objects are in contact, the minimum scaling $\alpha \leq 1$. The problem is formulated as:

$$\begin{aligned} & \underset{\alpha \in \mathbb{R}, x \in \mathbb{R}^3}{\text{minimize}} && \alpha \\ & \text{s.t.} && x \in S_1(\alpha), \\ & && x \in S_2(\alpha), \\ & && \alpha \geq 0, \end{aligned} \quad (4)$$

When the convex primitives are polytopes, the constraints are halfspace constraints of the convex hull.

IV. METHODOLOGY

A. Convex Decomposition

We decompose the collision-free space into a set of convex sets $\mathcal{C}_{free} = \mathcal{C}_1 \cup \mathcal{C}_2 \cup \dots \cup \mathcal{C}_L$ while keeping L as small as possible. For 2D problems, we use constrained Delaunay Triangulation [22] to split the non-convex free space into a number of triangles. It will output a list of valid triangles, and adjacent triangle pairs. We implement a greedy algorithm to merge adjacent triangles into larger convex polygons. We

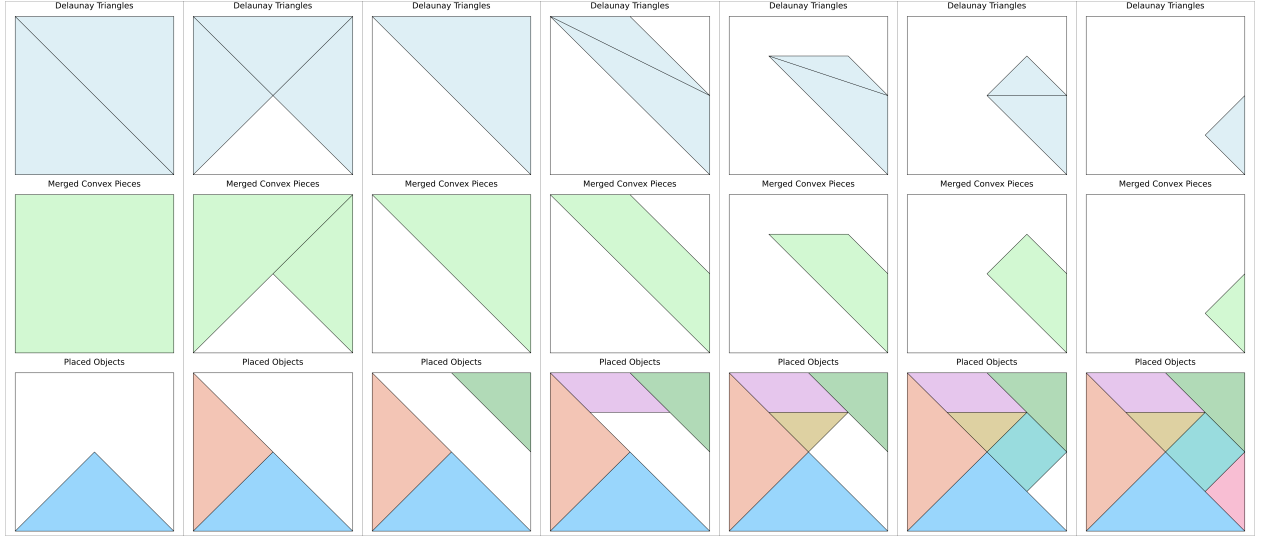


Fig. 2: Visualization of the Tangram puzzle with the given object sequence and cost functions. The first two rows show the free-space decomposition using Delaunay Triangulation [22] and merged convex polygons. At each iteration, we select the largest convex hull, and SQP will calculate the best placement in this convex hull, as shown in the third row.

will only merge the triangles/polygons when they share the same edge, and the potential merged polygon is convex, and the area of the merged polygon is large enough. The algorithm runs in a loop and will terminate when there are no adjacent polygons that can be merged. The complexity of the merging algorithm is $O(N^2)$, where N is the number of triangle pieces.

For 3D Packing, the 3D free space is represented as a 2D heightmap of size $M \times M$. We implement a greedy algorithm to partition the free space into a set of axis-aligned Maximal Empty Cuboids [23]. The algorithm sequentially scans the heightmap to locate uncovered cells. Upon finding a seed cell at height z , it executes a two-stage expansion along the X and Y axes to compute a maximal flat bounding region. This region is then extruded vertically to the maximum container height, forming a candidate 3D convex hull. To optimize execution, regions that fail minimum volume or width constraints are logged into a rejection buffer alongside valid hulls. This design ensures that each coordinate is evaluated as a candidate origin at most once, yielding an overall algorithmic complexity of $O(M^2)$.

B. Differentiable Collision Checking in Free Space

Theorem 1. A convex hull $\mathcal{A}$ is inside another convex hull $\mathcal{B}$ if and only if the vertices of $\mathcal{A}$ are inside $\mathcal{B}$.

Proof. Sufficiency: Given that $\mathcal{A} \subseteq \mathcal{B}$ and the vertices of $\mathcal{A}$ are a subset of $\mathcal{A}$, it is obvious that the vertices of $\mathcal{A}$ are a subset of $\mathcal{B}$.

Necessity: Given that the vertices of $\mathcal{A}$ are a subset of $\mathcal{B}$, we want to prove that any point in $\mathcal{A}$ also belongs to $\mathcal{B}$. Since the vertices of $\mathcal{A}$ are a convex combination of the vertices of $\mathcal{B}$, and any point in $\mathcal{A}$ is a convex combination of the vertices of $\mathcal{A}$. It is not difficult to calculate that any point in $\mathcal{A}$ can be expressed as a convex combination of the vertices of $\mathcal{B}$.

□

We consider each convex free space as a convex hull, which can be represented using $A \in \mathbb{R}^{k \times 3}$ and $b \in \mathbb{R}^k$ in a world frame $\mathbb{W}$ via $\mathcal{H}$ -representation. The number of halfspace constraints is k . Each object is defined with an attached body reference frame $\mathbb{B}$ with an origin $r \in \mathbb{R}^3$ expressed in a world frame $\mathbb{W}$. The vertices of the object can be expressed as $V_i \in \mathbb{R}^3, 1 \leq i \leq l$ in frame $\mathbb{B}$, where l is the number of vertices. The translation of the object can be defined as $t \in \mathbb{R}^3$. The orientation of an object is defined by a rotation matrix ${}^W Q^B \in \mathbb{R}^{3 \times 3}$ in the world frame, denoted as Q for shorthand.

Therefore, the vertices of the object can be expressed in $\mathbb{W}$ as $r + t + QV_i$. And we can express the constraint that each vertex lies inside the free space as:

$$A(r + t + QV_i) \leq b, 1 \leq i \leq l. \quad (5)$$

Then the problem of convex object placement can be written as:

$$\begin{aligned} & \underset{t, q \in \mathbb{R}^3}{\text{minimize}} && f(t, q) \\ & \text{s.t.} && A(r + t + Q(q)V_i) - b \leq 0, 1 \leq i \leq l. \end{aligned} \quad (6)$$

We denote $g_i(t, q) = A(r + t + Q(q)V_i) - b \in \mathbb{R}^k$. If we want to minimize the height of all the vertices in $\mathbb{W}$, we can choose

$$f(t, q) = \sum_{i=1}^l (t + Q(q)V_i)_z, \quad (7)$$

where the subscript z means to retrieve the third element of the position. Similarly, we can minimize a linear combination of the x, y, z positions of the sum of all vertices. The problem is nonlinear, as the mapping from the representation q to the rotation matrix Q is nonlinear.

C. Convex Hull Selection and Assignment

Now we can divide the free space into a set of convex hulls, and we can represent the constraint of placing each convex object inside each specific convex hull. However, the assignment between the convex hull and the convex object can result in different solutions. For example, if the volume of the convex hull is smaller than that of the convex object, there is no solution to place it inside. Iterating over all the convex hulls and solve the placement inside each convex hull is exhaustive but time-consuming. We rely on a set of heuristics to improve the solution quality and computational performance.

For the Tangram puzzle, we select the largest convex hull to place at each step. For 3D Bin Packing, at each step we sort the free-space convex hulls by height and volume. We first select the lowest and largest convex hull, then proceed to the next one. We iterate until we solve the placement optimization for each hull from the top k convex hulls. If there is no solution, we consider it a failure; otherwise, we select the best solution based on the objective function.

For non-convex object placement, it is more difficult to solve, as the different convex bodies of the object can belong to different convex hulls. A conservative estimate that treats the non-convex object as a single convex hull may work in simple scenarios, but it is not applicable to placement in narrow spaces, such as in the Tetris problem. As shown in Fig. 3, the free space is split into a set of convex hulls. We associate adjacent hulls and sort the resulting pairs by height. We solve for the top k lowest pairs separately. For each pair, we denote the convex hulls as $\mathcal{B}_1, \mathcal{B}_2$, and their halfspace constraints as $\bar{A}_1 \in \mathbb{R}^{k \times 3}, \bar{b}_1 \in \mathbb{R}^3$ and $\bar{A}_2 \in \mathbb{R}^{k \times 3}, \bar{b}_2 \in \mathbb{R}^3$. For the non-convex tetrominoes, we can split them into two convex bodies as $\mathcal{A}_1, \mathcal{A}_2$, and we can denote their vertices as $V_i, 1 \leq i \leq k_1$ and $V_i, k_1 < i \leq k_2$. Given these two pairs, we can have two assignments $\mathcal{A}_1 \subseteq \mathcal{B}_1, \mathcal{A}_2 \subseteq \mathcal{B}_2$ or $\mathcal{A}_1 \subseteq \mathcal{B}_2, \mathcal{A}_2 \subseteq \mathcal{B}_1$. We write the problem for the assignment $\mathcal{A}_1 \subseteq \mathcal{B}_1, \mathcal{A}_2 \subseteq \mathcal{B}_2$ as:

$$\begin{aligned} & \underset{t, q \in \mathbb{R}^3}{\text{minimize}} && f(t, q) \\ & \text{s.t.} && \bar{A}_1(r + t + Q(q)V_i) - \bar{b}_1 \leq 0, 1 \leq i \leq k_1 \\ & && \bar{A}_2(r + t + Q(q)V_i) - \bar{b}_2 \leq 0, k_1 < i \leq k_2. \end{aligned} \quad (8)$$

Here, we can adopt similar objective functions as in 7. If we choose the top k lowest adjacent hulls, we need to solve $2k$ different optimization problems.

D. Compute Analytic Derivative

We have framed the convex object placement in eq. 6 and Tetris placement in eq. 8. Given that the constraints and cost functions are written explicitly with respect to variables t, q , we can calculate the gradient of the cost function, the Jacobian of the constraints, and the Hessian of the Lagrangian function directly in closed form. We denote $x = [t, q] \in \mathbb{R}^6$ and consider the constraint $g_i = A(r + t + Q(q)V_i) - b$. We can write the Jacobians as:

$$\frac{\partial g_i}{\partial x} = \begin{bmatrix} \frac{\partial g_i}{\partial t} & \frac{\partial g_i}{\partial q} \end{bmatrix} = \begin{bmatrix} A & A \frac{\partial Q}{\partial q} V_i \end{bmatrix} \in \mathbb{R}^{k \times 6} \quad (9)$$

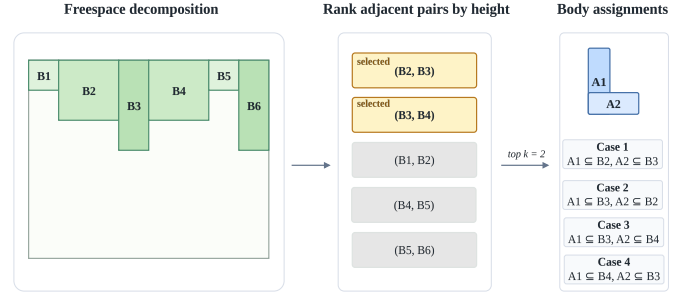


Fig. 3: Illustration of hull assignment for placing an L-shape object. The free-space convex hulls can be grouped into 5 pairs of adjacent hulls. We sort the pairs by height and select the top $k = 2$ hull pairs. Each convex body of the object is assigned to each hull, resulting in 4 combinations of constraint pairs, and we will solve these 4 problems separately.

where $\frac{\partial Q}{\partial q}$ is a third-order tensor and can be explicitly written as $\left(\frac{\partial Q}{\partial q}\right)_{ijk} = \frac{\partial Q_{ij}}{\partial q_k}$.¹

Since $g_i \in \mathbb{R}^k$, the second-order derivative of g_i with respect to x is a third-order tensor. To express it efficiently, we denote $g_{ij} = A_j(r + t + Q(q)V_i) - b_j \in \mathbb{R}, 1 \leq j \leq k$, where $A_j \in \mathbb{R}^{1 \times 3}$ and $b_j \in \mathbb{R}$ represent each halfspace constraint. Then we can calculate the Hessian as:

$$\frac{\partial^2 g_{ij}}{\partial x^2} = \begin{bmatrix} \frac{\partial^2 g_{ij}}{\partial t^2} & \frac{\partial^2 g_{ij}}{\partial t \partial q} \\ \frac{\partial^2 g_{ij}}{\partial q \partial t} & \frac{\partial^2 g_{ij}}{\partial q^2} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & A_j \frac{\partial^2 Q}{\partial q^2} V_i \end{bmatrix} \in \mathbb{R}^{6 \times 6} \quad (10)$$

where $A_j \frac{\partial^2 Q}{\partial q^2} V_i \in \mathbb{R}^{3 \times 3}$, $\frac{\partial^2 Q}{\partial q^2}$ is a fourth-order tensor and can be explicitly written as $\left(\frac{\partial^2 Q}{\partial q^2}\right)_{ijkl} = \frac{\partial^2 Q_{ij}}{\partial q_k \partial q_l}$.

We choose the Euler angle as the representation $q = [\alpha, \beta, \gamma]$ and follow the ZYX convention. We have $Q = R_z(\gamma)R_y(\beta)R_x(\alpha)$, where the rotation matrices can be written as:

$$\begin{aligned} R_z(\gamma) &= \begin{bmatrix} c_\gamma & -s_\gamma & 0 \\ s_\gamma & c_\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}, R_y(\beta) = \begin{bmatrix} c_\beta & 0 & s_\beta \\ 0 & 1 & 0 \\ -s_\beta & 0 & c_\beta \end{bmatrix}, \\ R_x(\alpha) &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha \\ 0 & s_\alpha & c_\alpha \end{bmatrix}, \end{aligned} \quad (11)$$

where c and s represent cos and sin functions. Rotation matrices have some good properties that we can use,

$$\begin{aligned} \frac{\partial R_x(\alpha)}{\partial \alpha} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & -s_\alpha & -c_\alpha \\ 0 & c_\alpha & -s_\alpha \end{bmatrix} = R_x(\alpha + \pi/2) - E_{11}, \\ \frac{\partial^2 R_x(\alpha)}{\partial \alpha^2} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & -c_\alpha & s_\alpha \\ 0 & -s_\alpha & -c_\alpha \end{bmatrix} = -R_x(\alpha) + E_{11}, \end{aligned} \quad (12)$$

¹ q_k means the k -th value of vector q and does not mean the covariant component.

where E_{ij} is the standard basis matrix that has value 1 at index i, j and 0 elsewhere. Similarly, we can have

$$\begin{aligned}\frac{\partial R_y(\beta)}{\partial \beta} &= R_y(\beta + \pi/2) - E_{22}, \quad \frac{\partial^2 R_y(\beta)}{\partial \beta^2} = -R_y(\beta) + E_{22}, \\ \frac{\partial R_z(\gamma)}{\partial \gamma} &= R_z(\gamma + \pi/2) - E_{33}, \quad \frac{\partial^2 R_z(\gamma)}{\partial \gamma^2} = -R_z(\gamma) + E_{33},\end{aligned}\quad (13)$$

Using the chain rule, we can compute

$$\begin{aligned}\frac{\partial Q}{\partial \alpha} &= R_z(\gamma)R_y(\beta)\frac{\partial R_x(\alpha)}{\partial \alpha} \\ &= R_z(\gamma)R_y(\beta)(R_x(\alpha + \pi/2) - E_{11}) \\ \frac{\partial^2 Q}{\partial \alpha^2} &= R_z(\gamma)R_y(\beta)\frac{\partial^2 R_x(\alpha)}{\partial \alpha^2} \\ &= R_z(\gamma)R_y(\beta)(-R_x(\alpha) + E_{11}) \\ \frac{\partial^2 Q}{\partial \alpha \partial \beta} &= R_z(\gamma)\frac{\partial R_y(\beta)}{\partial \beta}\frac{\partial R_x(\alpha)}{\partial \alpha} \\ &= R_z(\gamma)(R_y(\beta + \pi/2) - E_{22})(R_x(\alpha + \pi/2) - E_{11}) \\ \frac{\partial^2 Q}{\partial \alpha \partial \gamma} &= \frac{\partial R_z(\gamma)}{\partial \gamma}R_y(\beta)\frac{\partial R_x(\alpha)}{\partial \alpha} \\ &= (R_z(\gamma + \pi/2) - E_{33})R_y(\beta)(R_x(\alpha + \pi/2) - E_{11})\end{aligned}\quad (14)$$

Similarly, we can compute $\frac{\partial Q}{\partial \beta}$, $\frac{\partial Q}{\partial \gamma}$, $\frac{\partial^2 Q}{\partial \beta^2}$, $\frac{\partial^2 Q}{\partial \gamma^2}$, $\frac{\partial^2 Q}{\partial \beta \partial \gamma}$. At every iteration when we have the values of α, β, γ , we first compute the rotation matrices $R_x(\alpha)$, $R_x(\alpha + \pi/2)$, $R_y(\beta)$, $R_y(\beta + \pi/2)$, $R_z(\gamma)$, $R_z(\gamma + \pi/2)$, then we can calculate $\frac{\partial Q}{\partial q}$ and $\frac{\partial^2 Q}{\partial q^2}$ by composing these rotation matrices following eq. 14.

If we choose the cost function f as in eq. 7, we can calculate the gradient of the cost function as

$$\begin{aligned}\nabla f &= \begin{bmatrix} \nabla_t f \\ \nabla_q f \end{bmatrix} \in \mathbb{R}^6, \\ \nabla_t f &= l \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ l \end{bmatrix}, \\ \nabla_q f &= \begin{bmatrix} \frac{\partial Q_{31}}{\partial q} & \frac{\partial Q_{32}}{\partial q} & \frac{\partial Q_{33}}{\partial q} \end{bmatrix} \sum_{i=1}^l V_i \\ &= \bar{a} \frac{\partial Q_{31}}{\partial q} + \bar{b} \frac{\partial Q_{32}}{\partial q} + \bar{c} \frac{\partial Q_{33}}{\partial q} \in \mathbb{R}^3,\end{aligned}\quad (15)$$

where the constant $\sum_{i=1}^l V_i$ is denoted as $[\bar{a}, \bar{b}, \bar{c}]^T$. Then we can calculate the Hessian of the cost function as

$$\begin{aligned}\nabla_{xx}^2 f &= \begin{bmatrix} 0 & 0 \\ 0 & \nabla_{qq}^2 f \end{bmatrix} \in \mathbb{R}^{6 \times 6}, \\ \nabla_{qq}^2 f &= \bar{a} \frac{\partial^2 Q_{31}}{\partial q^2} + \bar{b} \frac{\partial^2 Q_{32}}{\partial q^2} + \bar{c} \frac{\partial^2 Q_{33}}{\partial q^2} \in \mathbb{R}^{3 \times 3}.\end{aligned}\quad (16)$$

We use $c(x)$ and $J(x)$ to denote the whole constraints and Jacobian matrix of the constraints,

$$c(x) = \begin{bmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_l(x) \end{bmatrix} \in \mathbb{R}^{kl}, \quad J(x) = \begin{bmatrix} \frac{\partial g_1}{\partial x} \\ \frac{\partial g_2}{\partial x} \\ \vdots \\ \frac{\partial g_l}{\partial x} \end{bmatrix} \in \mathbb{R}^{kl \times 6}, \quad (17)$$

The Lagrangian function for this problem is

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T c(x), \quad (18)$$

where $\lambda \in \mathbb{R}^{kl}$ is the lagrangian multiplier. We can compute the Hessian of the Lagrangian function as:

$$\begin{aligned}\nabla_{xx}^2 \mathcal{L}(x, \lambda) &= \nabla_{xx}^2 f + \sum_{i=1}^l \sum_{j=1}^k \lambda_{ij} \frac{\partial^2 g_{ij}}{\partial x^2} = \begin{bmatrix} 0 & 0 \\ 0 & \nabla_{qq}^2 \mathcal{L} \end{bmatrix}, \\ \nabla_{qq}^2 \mathcal{L}(x, \lambda) &= \nabla_{qq}^2 f + \sum_{i=1}^l \sum_{j=1}^k \lambda_{ij} \frac{\partial^2 g_{ij}}{\partial q^2} \in \mathbb{R}^{3 \times 3}\end{aligned}\quad (19)$$

where $\nabla_{xx}^2 f$ and $\frac{\partial^2 g_{ij}}{\partial x^2}$ can be calculated using eq. 16 and eq. 10. To make the calculations more efficient, we use the Kronecker product and vectorization to eliminate for loops when computing the analytic derivatives. The memory is entirely stack-allocated and one-shot during initialization without tedious forward or backward calculations in automatic differentiation.

E. Object Placement Optimization via SQP

We solve the nonlinear optimization problem 6 and 8 via Sequential Quadratic Programming [24]. The idea is to model the optimization problem at the current iterate (x_n, λ_n) as a quadratic programming subproblem, then use its solution to obtain a new iterate (x_{n+1}, λ_{n+1}) . Suppose at iterate (x_n, λ_n) , the quadratic problem is modeled as:

$$\begin{aligned}\text{minimize}_{p \in \mathbb{R}^6} \quad & f_n + \nabla f_n^T p + \frac{1}{2} p^T \nabla_{xx}^2 \mathcal{L}_n p \\ \text{s.t.} \quad & J_n p + c_n \leq 0\end{aligned}\quad (20)$$

where J_n and c_n are the Jacobian matrix and constraints at x_n defined in eq. 17.

We use Clarabel [8] to solve the QP problem. Note that in eq. 19, $\nabla_{xx}^2 \mathcal{L}_n$ is zero everywhere except the bottom right block $\nabla_{qq}^2 \mathcal{L}_n$, and $\nabla_{qq}^2 \mathcal{L}_n$ is possibly indefinite. We add a scaled identity matrix to $\nabla_{xx}^2 \mathcal{L}_n$ to improve numerical stability. We perform a backtracking line search to calculate how far we should move along the direction p and enforce a sufficient decrease in the merit function following the Armijo rule. We choose the merit function $\phi(x, \mu) = f(x) + \mu^T c(x)$, where $\mu \in \mathbb{R}^{kl}$ is the penalty parameter. The penalty parameter is zero when the corresponding constraint is smaller than the threshold tol ; otherwise, it is set to a positive constant. The SQP terminates when the norm of the current step $\|p\|$ and the constraint violation $\max(c_n)$ are smaller than the tolerance threshold tol . We set $tol = 1 \times 10^{-4}$ and the maximum SQP iteration number to 50.

Like many other optimization methods, SQP is also a local-optimal solver, making it difficult to find global optimal solutions in one shot. In our placement setting, global optimality is essential: a local optimal solution may have zero constraint violation, yet its failure to achieve the best placement leaves no room for the following objects to be placed. We thus parallelize the SQP solver using randomized initial states and select the best placement based on the merit function.

TABLE I: Computation time for calculating gradient, Jacobian, and Hessian.

	scenario	gradient (ns)	Jacobian (ns)	Hessian (ns)
Analytic	2D	16.29	24.96	41.45
	3D	90.79	188.70	368.59
ForwardDiff	2D	21.16	45.70	311.65
	3D	167.82	345.27	7652.00

V. RESULTS

We implement our algorithms in Julia and leverage BenchmarkTools [25] to efficiently run all our modules multiple times to evaluate computational performance. We run experiments on a computer with an AMD Ryzen 9 7950x3D 16-core CPU.

A. Analytic Derivatives vs. AutoDiff

We first compare the computational performance between analytic derivatives and ForwardDiff [26]. During each run, we randomize the object’s state in 2D and 3D scenarios and compute the gradient of the cost function, the Jacobian of the constraints, and the Hessian of the Lagrangian. We report the median computation time in Table I. At every run, we check that the values of the analytic derivatives are identical to the values of ForwardDiff. The computation time of analytic derivatives is consistently better than ForwardDiff in 2D and 3D scenarios, especially for the most time-consuming Hessian calculation; the analytic solution is almost 10 – 20 times faster.

B. Solve Tangram Puzzle

We evaluate our method on a Tangram puzzle as shown in Fig. 2. Here, the object sequence and corresponding cost functions are given, and we aim to find the optimal placement at each time step while minimizing or maximizing the objects’ height or width. We compare SOPO-CD to the same SQP with Differentiable Collision (DCOL) [14], where DCOL uses ForwardDiff to calculate the constraint Jacobians and Hessian of the Lagrangian by default. We report the median computation time and average converged SQP iterations for placing each object in Table II. For each object, the convex decomposition preprocessing takes 25–75 μ s. With convex decomposition, SQP shows consistent performance, takes less time (0.2 – 0.8ms per object), and requires fewer iterations to converge. Compared to us, for SQP with DCOL, the computation time increases as the number of objects grows, because the number of collision pairs between the current object and the placed objects increases. As a consequence, it is very challenging for DCOL

TABLE II: Computation time per step for Tangram.

		obj 1	obj 2	obj 3	obj 4	obj 5	obj 6	obj 7
SOPO-CD	Preprocess (μ s)	35.34	60.35	25.05	75.44	49.64	39.94	31.78
	SQP Time (ms)	0.84	0.18	0.19	0.26	0.36	0.25	0.26
	SQP Iters	12.53	3.86	4.21	4.71	11.00	4.00	6.23
DCOL	SQP Time (ms)	0.87	1.02	0.97	3.43	15.67	17.63	20.76
	SQP Iters	12.68	13.49	6.94	27.31	33.96	49.91	50.00

TABLE III: Performance for full sequence Tangram.

	n.threads	total time (ms)	successful placements	memory estimate (MiB)
SOPO-CD	2	9.02	4.35	2.53
	4	9.31	5.62	4.66
	8	10.02	6.63	9.44
	16	12.98	6.96	17.67
DCOL	2	27.48	2.63	3.26
	4	46.34	3.54	6.76
	8	91.79	4.53	18.26
	16	123.58	4.95	61.95

to place the last two objects, since the SQP iterations hit the maximum limit of 50.

We also report the median total time, average number of successful placements, and memory estimates for placing the full sequence of objects in Table III. SOPO-CD consistently performs better than SQP with DCOL. When setting the number of threads to 8, our method can place almost all 7 objects successfully with a total computation time around 10ms.

C. Solve 2D Tetris Puzzle

We evaluate our method on the 5-Tetris and 8-Tetris benchmarks introduced by cuTAMP [20] and SPaSM [21]. The goal is to determine feasible object poses for the 5 and 8 objects so that they fit within a constrained bounding box. We compare with SPaSM without trajectory optimization. The results of our method are shown in Table IV. The total time reports the total average time for convex decomposition, constraint extraction, and batch SQP calculations. The total time for 5-Tetris is around 3 – 5ms. The total time for 8-Tetris is around 7 – 11ms. The success rate and number of successful placements increase with the number of threads, and it is almost always successful when setting the number of threads to 8.

We run SPaSM on 5-Tetris and 8-Tetris on an NVIDIA GeForce RTX 4090 GPU. The results of SPaSM are shown in Table V. We keep the remaining hyperparameters unchanged and test only on different termination cost thresholds. The default cost thresholds for 5-Tetris and 8-Tetris are 0.42 and 0.66, respectively. In these settings, the algorithms converge within 1 and 10ms, with an average penetration of the wall and sphere between 2×10^{-4} and 3×10^{-3} . We also observe that lowering the cost threshold can improve the penetration distance, but SPaSM will not converge when the threshold is much lower. In contrast, our method can handle hard constraints with a tolerance of 1×10^{-4} .

We generalize our method to a full Tetris problem, where we include all 7 different tetrominoes, as shown in Fig. 4. The objective function for each placement is $f(t, q) =$

TABLE IV: Performance for mini Tetris using SOPO-CD.

	n.threads	total time (ms)	successful placements	success rate (%)
5-Tetris	2	3.14	4.75	74.95
	4	3.46	4.94	93.71
	8	4.62	4.99	99.84
8-Tetris	2	10.45	7.28	35.13
	4	6.90	7.80	81.05
	8	8.22	7.99	98.71

TABLE V: Performance for mini Tetris using SPaSM [21].

	cost.thresh	total time (ms)	wall penetration	sphere penetration	success rate (%)
5-Tetris	0.42	0.22	3.10e-3	3.00e-4	100
	0.38	3.52	3.08e-3	2.86e-4	100
	0.37	95.11	2.83e-3	3.18e-4	100
	≤ 0.36	-	-	-	0
8-Tetris	0.66	6.84	2.93e-3	2.34e-4	100
	0.64	14.15	2.89e-3	2.08e-4	100
	0.63	55.78	2.97e-3	1.86e-4	100
	≤ 0.62	-	-	-	0

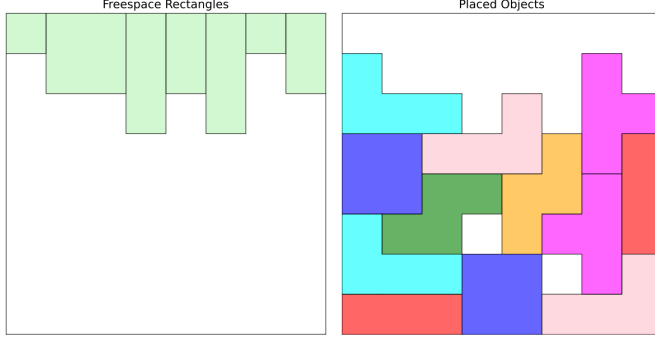


Fig. 4: An intermediate screenshot of convex hull generation and placed objects for 2D Tetris.

$\sum_{i=1}^l (t + Q(q)V_i)_x + 5 \sum_{i=1}^l (t + Q(q)V_i)_y$, which optimizes the object to be at the bottom-left area. We test our method against a grid search algorithm on a number of random sequences. We show the results of grid search on different grid sizes in Table VI. We show the results of SOPO-CD on different top k convex pairs in Table VII. At each iteration, the preprocessing takes around 6 μ s and solving $2k$ placement optimization problems for each object takes 10 – 20ms. As the solver batch size and top- k increase, the number of successful placements and the occupancy rate rise, reaching 77% when setting the number of threads to 8 and $k = 1$. The computation time of SOPO-CD is consistently performant, and it is 50 times faster than grid search at the largest grid size.

D. Solve 3D Bin Packing

We evaluate our method on the 3D Bin Packing problem and compare it with the standard grid search algorithm on the heightmap. The container size is $[5, 5, 5]$, and the packed objects are 3D rectangular shapes with sizes $[3, 2, 1]$, $[2, 2, 2]$, $[1, 2, 2]$, $[1, 1, 2]$. We use the heightmap to

TABLE VI: Performance for full Tetris using grid search.

	grid size	compute time per obj (ms)	successful placements	occupancy rate (%)
Grid-Search	8×8	0.03	14.10	84.86
	80×80	1.69	13.97	83.92
	800×800	1455.24	13.67	82.29

TABLE VII: Performance for full Tetris.

	top k	preprocess per obj (μ s)	compute time per obj (ms)	successful placements	occupancy rate (%)
SOPO-CD (n.threads=2)	1	5.40	9.05	9.95	59.46
	2	5.59	13.52	10.88	65.13
	3	5.83	18.70	11.06	66.23
SOPO-CD (n.threads=4)	1	5.73	12.39	12.06	72.28
	2	6.09	19.06	12.28	73.62
	3	5.84	26.05	12.32	73.94
SOPO-CD (n.threads=8)	1	5.58	16.19	12.75	76.56
	2	6.27	24.02	12.61	75.66
	3	6.39	34.31	12.79	76.79

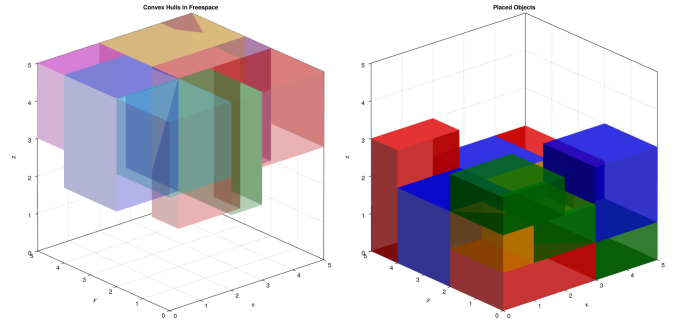


Fig. 5: An intermediate screenshot of convex hull generation and placed objects for 3D Bin Packing.

represent the occupied objects. The heightmap is characterized by a predefined grid size $M \times M$. For every object placement, we use the same objective function as the grid search algorithm, $f(t, q) = t_x + t_y + 5t_z$, where $t = [t_x, t_y, t_z]$ is the center of the cuboid. This is the same heuristic as in DBLF [12]. The time complexity for the grid search algorithm is $O(M^2)$.

We test SOPO-CD against the grid search algorithm on a number of random sequences of objects. The convex hull generation and placed objects at an intermediate step are visualized in Fig. 5. The quantitative results are shown in Table VIII. We compare the average preprocessing and computation times for each object, the number of successful placements across the whole sequence, and the occupancy rate for all placements. The preprocessing includes convex hull generation and constraint extraction. It is shown that the search time grows drastically as the grid size increases. The preprocessing time for our method also increases with grid size, but remains within 1ms. Our method generates a set of axis-aligned convex hulls, and we select the 3 lowest convex hulls and run the batched SQP solver on each hull. The computation time of our method shows consistent performance across varying grid sizes and grows only linearly with the solver batch size and the number of selected convex hulls. At the largest grid size, our method is more than 200 times faster than the grid search algorithm. The number of successful placements and the occupancy rate also increase significantly as the solver batch size increases, and the performance is competitive with that of the grid search.

TABLE VIII: Performance for 3D Bin Packing.

	grid size	preprocess per obj (μ s)	compute time per obj (ms)	successful placements	occupancy rate (%)
Grid Search	10×10	-	0.01	21.80	85.32
	50×50	-	0.41	21.93	85.47
	100×100	-	6.28	22.04	86.63
	250×250	-	222.60	21.70	85.47
	500×500	-	3940.55	22.20	85.97
SOPO-CD (n.threads=2)	10×10	1.46	7.32	16.17	62.02
	50×50	9.36	7.49	17.02	64.42
	100×100	31.83	7.22	16.02	62.84
	250×250	189.92	7.67	16.20	64.20
	500×500	666.90	8.02	16.42	64.53
SOPO-CD (n.threads=4)	10×10	1.57	10.74	19.77	76.92
	50×50	9.66	10.78	19.60	77.25
	100×100	33.32	10.64	18.53	74.73
	250×250	180.40	10.08	19.94	76.38
	500×500	694.34	10.93	19.94	77.86
SOPO-CD (n.threads=8)	10×10	1.69	19.90	20.57	81.00
	50×50	9.52	14.31	20.45	81.31
	100×100	31.21	14.04	21.33	81.72
	250×250	176.49	13.92	22.18	83.30
	500×500	692.78	14.68	20.30	80.26

E. Real-world Experiment

For the real-world experiment, we deploy our algorithm to solve the Tangram puzzle using an Allegro hand and an Xarm, as shown in Fig. 1. The system runs in an open loop; once the object's goal location is calculated, it is fed into the motion planner to perform pick-and-place. We predefine the contact points for each object and the finger, and run an online optimization to compute the force closure. We plan and control the arm using an off-the-shelf sampling-based motion planner [27].

VI. CONCLUSION

We introduced SOPO-CD, a sequential optimization framework that solves the object placement problem with differentiable collision constraints in free space. We divided the collision-free space into various convex hulls and assigned the convex hull to each convex body. We implemented a custom SQP solver and solved the optimal placement in milliseconds. We validated our method on a set of tasks, including the 2D Tangram puzzle, 2D Tetris puzzle, and 3D Bin Packing, and demonstrated strong computational performance and packing utility. We also demonstrated solving a real-world Tangram puzzle using an Allegro Hand and an Xarm.

Despite these strengths, some limitations remain to be addressed in the future. The convex hull selection is not exhaustive, and our 3D convex decomposition module generates axis-aligned cuboids; a more general method is needed to handle more complex scenarios. Our algorithm is greedy in choosing the current placement without looking ahead to the future. The objective functions/heuristics for each object are predefined, and it is interesting to combine tree search and backtracking over different combinations of heuristics as in [28], [29].

ACKNOWLEDGMENT

This project has received funding from the European Union's Horizon Europe programme under Grant Agreement No. 101120823, project MANiBOT.

REFERENCES

- [1] F. Wang and K. Hauser, "Stable bin packing of non-convex 3d objects with a robot manipulator," in *IEEE International Conference on Robotics and Automation*, 2019, pp. 8698–8704.
- [2] H. Zhao, Q. She, C. Zhu, Y. Yang, and K. Xu, "Online 3d bin packing with constrained deep reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, 2021, pp. 741–749.
- [3] N. Funk, G. Chalvatzaki, B. Belousov, and J. Peters, "Learn2assemble with structured representations and search for robotic architectural construction," in *Conference on Robot Learning*, 2022.
- [4] H. Zhao, Z. Pan, Y. Yu, and K. Xu, "Learning physically realizable skills for online packing of general 3d shapes," *ACM Transactions on Graphics*, vol. 42, no. 5, pp. 1–21, 2023.
- [5] S. Yang, S. Song, S. Chu, R. Song, J. Cheng, Y. Li, and W. Zhang, "Heuristics integrated deep reinforcement learning for online 3d bin packing," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 1, pp. 939–950, 2023.
- [6] H. Zhao, Y. Yu, and K. Xu, "Learning efficient online 3d bin packing on packing configuration trees," in *International conference on learning representations*, 2021.
- [7] H. Zhao, J. Xu, K. Yu, R. Hu, C. Zhu, B. Du, and K. Xu, "Deliberate planning of 3d bin packing on packing configuration trees," *The International Journal of Robotics Research*, 2025.
- [8] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives," *arXiv preprint arXiv:2405.12762*, 2024.
- [9] S. Martello, D. Pisinger, and D. Vigo, "The three-dimensional bin packing problem," *Operations research*, vol. 48, no. 2, pp. 256–267, 2000.
- [10] B. S. Baker, E. G. Coffman, Jr, and R. L. Rivest, "Orthogonal packings in two dimensions," *SIAM Journal on computing*, vol. 9, no. 4, pp. 846–855, 1980.
- [11] D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, and R. L. Graham, "Worst-case performance bounds for simple one-dimensional packing algorithms," *SIAM Journal on computing*, vol. 3, no. 4, pp. 299–325, 1974.
- [12] K. Karabulut and M. M. İnceoğlu, "A hybrid genetic algorithm for packing in 3d with deepest bottom left with fill method," in *International Conference on Advances in Information Systems*. Springer, 2004, pp. 441–450.
- [13] L. Montaut, Q. Le Lidec, A. Bambade, V. Petrik, J. Sivic, and J. Carpentier, "Differentiable collision detection: a randomized smoothing approach," in *IEEE International Conference on Robotics and Automation*, 2023, pp. 3240–3246.
- [14] K. Tracy, T. A. Howell, and Z. Manchester, "Differentiable collision detection for a set of convex primitives," in *IEEE International Conference on Robotics and Automation*, 2023, pp. 3663–3670.
- [15] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *International Workshop on the Algorithmic Foundations of Robotics*. Springer, 2015, pp. 109–124.
- [16] S. Liu, M. Watterson, K. Mohta, K. Sun, S. Bhattacharya, C. J. Taylor, and V. Kumar, "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments," *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1688–1695, 2017.
- [17] J. Tordesillas, B. T. Lopez, M. Everett, and J. P. How, "Faster: Fast and safe trajectory planner for navigation in unknown environments," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 922–938, 2021.
- [18] T. Marucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science robotics*, vol. 8, no. 84, p. eadf7843, 2023.
- [19] Z. Yang, J. Mao, Y. Du, J. Wu, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Compositional Diffusion-Based Continuous Constraint Solvers," in *Conference on Robot Learning*, 2023.
- [20] W. Shen, C. Garrett, N. Kumar, A. Goyal, T. Hermans, L. P. Kaelbling, T. Lozano-Pérez, and F. Ramos, "Differentiable gpu-parallelized task and motion planning," in *Robotics: Science and Systems*, 2025.
- [21] L. Chen, S. R. Iyer, and Z. Kingston, "Differentiable particle optimization for fast sequential manipulation," *arXiv preprint arXiv:2510.07674*, 2025.
- [22] D. J. VandenHeuvel, "DelaunayTriangulation.jl: A Julia package for Delaunay triangulations and Voronoi tessellations in the plane," *Journal of Open Source Software*, vol. 9, no. 101, p. 7174, Sep. 2024.
- [23] S. C. Nandy and B. B. Bhattacharya, "Maximal empty cuboids among points and blocks," *Computers & Mathematics with Applications*, vol. 36, no. 3, pp. 11–20, 1998.
- [24] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 2006.
- [25] J. Chen and J. Revels, "Robust benchmarking in noisy environments," *arXiv preprint arXiv:1608.04295*, 2016.
- [26] J. Revels, M. Lubin, and T. Papamarkou, "Forward-mode automatic differentiation in Julia," *arXiv:1607.07892 [cs.MS]*, 2016.
- [27] I. A. Sucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.
- [28] M. Toussaint and M. Lopes, "Multi-bound tree search for logic-geometric programming in cooperative manipulation domains," in *IEEE International Conference on Robotics and Automation*, 2017, pp. 4044–4051.
- [29] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.